Universidade Católica de Brasília
Pró-Reitoria Acadêmica
Curso de Engenharia de Software

SISTEMA DE GESTÃO DE OFICINA MECÂNICA

Projeto Final da Disciplina – Etapa 2

Relatório Técnico: Operação, Interface e Desempenho (N2)

Equipe de Desenvolvimento:

João Lucas Claver de Oliveira – UC25101220

Gustavo Pereira Rodrigues – UC25102347

Igor da Nóbrega Loureiro – UC25102776

Danillo de Carvalho Montalvão – UC25101296

Eduardo Mourão Aragão – UC25200937

Professor: Samuel Novais Moura Júnior

Disciplina: Laboratório de Banco de Dados

Brasília - DF, 2026

1. Introdução à Operação do Sistema (Etapa 2)

Este documento consolida a Etapa 2 do projeto do Sistema de Gestão de Oficina Mecânica. Com o modelo lógico relacional normalizado e o banco de dados implementado em MySQL (Etapa 1), esta fase foca na operação do sistema por meio de uma aplicação desenvolvida em Python, com interface em Streamlit. A aplicação exercita as regras de negócio implementadas no banco de dados.

Este relatório documenta exclusivamente o que está implementado. Melhorias que ainda não foram desenvolvidas são identificadas como propostas futuras.

1.1 Tecnologias e Estrutura da Aplicação

Componente	Descrição
MySQL 8.0+	Sistema gerenciador de banco de dados.
Python 3.10+ e Streamlit	Linguagem e framework da interface interativa.
mysql-connector-python	Driver de acesso ao MySQL.
python-dotenv	Leitura das credenciais a partir do arquivo .env.
app.py	Interface Streamlit da aplicação.
database.py	Função get_connection(), que abre a conexão com o MySQL usando as variáveis do .env.
.env.example	Modelo de configuração, com DB_HOST, DB_USER, DB_PASSWORD e DB_NAME.

2. Aplicação e Camada de Acesso a Dados (B1 e B2)

A aplicação foi construída com foco na integridade das operações. O acesso aos dados é feito por meio de consultas parametrizadas, e as regras críticas de histórico, estoque e valor total são executadas por triggers no próprio MySQL, e não pela aplicação.

2.1 Funcionalidades Implementadas

Entidade / fluxo	Operações disponíveis na aplicação
CLIENTE	CRUD (cadastro, consulta, atualização e exclusão)
VEICULO	CRUD
PECA	CRUD
MECANICO	CRUD
ORDEM_SERVICO	Abertura transacional (criação da OS e inclusão de um item de peça) e consulta/listagem por status

A entidade ORDEM_SERVICO **não possui CRUD completo**: a aplicação não oferece telas de atualização, conclusão, cancelamento ou exclusão de OS. Os status válidos da OS são Orçamento, Aprovado, Em Execução, Concluído e Cancelado. Uma OS concluída exige aprovação, data de conclusão e forma de pagamento. Quanto aos dados de clientes, o CPF é armazenado em PESSOA_FISICA e o CNPJ em PESSOA_JURIDICA; a tabela CLIENTE não possui coluna cpf.

2.2 Tratamento de Erros e Restrições

As restrições definidas no banco de dados (chaves estrangeiras, constraints UNIQUE e CHECK) são tratadas pela aplicação para produzir retornos mais claros ao usuário. Por exemplo, ao tentar excluir um registro que possui vínculos (violação de restrição de chave estrangeira), o erro do driver MySQL é interceptado e a interface exibe uma mensagem informativa, em vez do erro bruto:

Aviso de Interface: "Não é possível excluir o registro, pois existem dependências vinculadas a ele."

2.3 Defesa contra SQL Injection

As consultas que recebem entrada do usuário utilizam placeholders `%s` e passam os valores separadamente, por meio de `cursor.execute(query, parametros)`, sem concatenação de strings. Assim, uma entrada como `' OR 1=1 --` é tratada como valor literal, e não como parte do comando SQL. O exemplo abaixo, ilustrativo, usa a busca por CPF, que pertence à tabela PESSOA_FISICA:

```
cursor = conexao.cursor()
query = "SELECT cpf, rg, data_nasc FROM PESSOA_FISICA WHERE cpf = %s"

# A entrada maliciosa é enviada como parâmetro, não como comando SQL.
entrada_usuario = "' OR 1=1 --"
cursor.execute(query, (entrada_usuario,))
resultado = cursor.fetchall()   # nenhum CPF corresponde ao texto informado
```

As credenciais de acesso ao SGBD não ficam no código-fonte. Elas são lidas do arquivo `.env` pela função `get_connection()` (arquivo `database.py`). O `.env` é de uso local e está listado no `.gitignore`, portanto não é versionado; o repositório contém apenas o `.env.example`, com os nomes das variáveis.

3. Transações e Controle de Concorrência (B3)

O evento transacional implementado na aplicação é a abertura de uma Ordem de Serviço (OS). Nela, duas operações devem ocorrer de forma atômica: a criação da OS e a inclusão de um item de peça. Ou ambas são efetivadas, ou nenhuma é aplicada.

3.1 Operação Transacional: Abertura de OS

A sequência executada pela aplicação é: (1) criar a ORDEM_SERVICO; (2) adicionar um item em ITEM_PECA; (3) efetivar com COMMIT; (4) em caso de erro, executar ROLLBACK. Em SQL, o fluxo é:

```
START TRANSACTION;

-- 1. Cria a Ordem de Serviço
INSERT INTO ORDEM_SERVICO (data_abertura, data_previsao, status, id_veiculo, id_mecanico)
VALUES (NOW(), %s, %s, %s, %s);

-- 2. Adiciona um item de peça à OS recém-criada
INSERT INTO ITEM_PECA (id_os, id_peca, quantidade, valor_cobrado)
VALUES (LAST_INSERT_ID(), %s, %s, %s);

COMMIT;   -- ou ROLLBACK em caso de erro
```

Na aplicação Python, o bloco é controlado pela conexão do *mysql-connector-python*:

```
try:
    conexao.start_transaction()
    cursor.execute(sql_insert_os, parametros_os)
    id_os = cursor.lastrowid
    cursor.execute(sql_insert_item_peca, (id_os, id_peca, quantidade, valor_cobrado))
    conexao.commit()
except mysql.connector.Error:
    conexao.rollback()
    raise
```

Papel dos triggers. A aplicação não insere o histórico, não atualiza o estoque e não recalcula o valor total manualmente. Essas ações são realizadas automaticamente pelos triggers do banco: a abertura da OS gera o registro inicial em HISTORICO_STATUS_OS; a inserção do item valida a disponibilidade em estoque e baixa a quantidade em PECA; e o valor total da OS é calculado a partir dos itens.

Comportamento em caso de falha. Se uma das etapas gerar erro, por exemplo estoque insuficiente rejeitado pelo trigger na inserção do item, a exceção é capturada e a aplicação executa o ROLLBACK. Dessa forma, a OS criada na etapa 1 não é persistida sem o respectivo item.

3.2 Experimento de Concorrência

O cenário abaixo, de natureza conceitual, descreve o comportamento esperado do InnoDB quando duas sessões (atendentes) inserem itens da mesma peça simultaneamente. Como a baixa de estoque é realizada pelo trigger sobre a linha da peça, a segunda sessão aguarda a liberação do bloqueio da primeira.

Tempo	Sessão A (Atendente 1)	Sessão B (Atendente 2)
T1	START TRANSACTION;	START TRANSACTION;
T2	INSERT INTO ITEM_PECA (..., id_peca = 5, ...); (o trigger atualiza a linha de PECA e obtém bloqueio exclusivo)	
T3		INSERT INTO ITEM_PECA (..., id_peca = 5, ...); (aguarda a liberação do bloqueio)
T4	COMMIT; (libera o bloqueio)	
T5		Prossegue com o estoque já atualizado: o trigger valida a quantidade e a sessão efetiva COMMIT ou recebe erro, se o estoque for insuficiente.

Justificativa do Isolamento: o MySQL (InnoDB) opera por padrão em REPEATABLE READ, que oferece leituras consistentes dentro da transação. As escritas, por sua vez, usam bloqueios de linha, o que serializa as atualizações concorrentes sobre o mesmo registro e preserva a consistência do estoque. Este cenário descreve o comportamento esperado e não representa uma medição realizada pela equipe.

4. Otimização de Consultas (B4)

Esta seção lista os índices existentes no banco de dados e a finalidade de cada um. Não foram coletadas métricas de EXPLAIN, Query Cost ou tempo de execução nesta etapa; por isso, o relatório não apresenta custos ou tempos comparativos.

4.1 Índices Existentes

Índice	Tabela (coluna)	Finalidade
uq_veiculo_placa	VEICULO (placa)	Índice gerado pela restrição UNIQUE; garante placa única e atende buscas por placa.
idx_os_id_veiculo	ORDEM_SERVICO (id_veiculo)	Junções e consultas das OS de um veículo.
idx_os_id_mecanico	ORDEM_SERVICO (id_mecanico)	Junções e consultas das OS de um mecânico.
idx_os_status	ORDEM_SERVICO (status)	Consulta e listagem de OS por status.
idx_item_servico_id_servico	ITEM_SERVICO (id_servico)	Junções com a tabela de serviços.
idx_item_peca_id_peca	ITEM_PECA (id_peca)	Junções com a tabela de peças.
idx_historico_data_mudanca	HISTORICO_STATUS_OS (data_mudanca)	Consultas do histórico por data de mudança.

Exemplo de consulta presente na aplicação, que se beneficia de `idx_os_status`:

```
SELECT id_os, data_abertura, data_previsao, status, valor_total
FROM ORDEM_SERVICO
WHERE status = %s
ORDER BY data_abertura DESC;
```

4.2 Propostas Futuras

Os itens abaixo **não estão implementados** e constituem propostas para evolução do sistema:

- **Índice FULLTEXT em PECA.descricao:** buscas textuais com `LIKE '%termo%'` não aproveitam índices comuns. O índice FULLTEXT exigiria adaptar a consulta para `MATCH ... AGAINST`.
- **Índice por data de conclusão:** um índice em ORDEM_SERVICO(data_conclusao) poderia apoiar relatórios por período.
- **Testes de desempenho:** executar EXPLAIN sobre as consultas mais frequentes com volume ampliado de dados sintéticos e registrar métricas reais antes de criar novos índices.

5. Segurança e Controle de Acesso (B5)

5.1 Medidas Implementadas

- Credenciais do SGBD mantidas no arquivo `.env`, de uso local e não versionado no Git.
- Consultas com entrada do usuário parametrizadas por placeholders `%s`.
- Restrições de integridade do banco de dados (chaves estrangeiras, UNIQUE e CHECK) e triggers de regras de negócio.

A aplicação atual **não possui** login, autenticação ou autorização por perfil, e não utiliza roles ou usuários MySQL distintos por perfil de acesso.

5.2 Recomendações Futuras

Em aderência ao princípio do menor privilégio (RN20), recomenda-se, em etapas futuras:

- criar perfis de acesso (roles) no MySQL: administração com acesso total; atendimento com SELECT, INSERT e UPDATE sobre CLIENTE, VEICULO e ORDEM_SERVICO, sem DELETE; e mecânico com leitura restrita;
- disponibilizar ao mecânico uma view que exponha apenas as OS sob sua responsabilidade, sem dados financeiros sigilosos, com o filtro por mecânico aplicado pela aplicação;
- implementar autenticação na aplicação, armazenando senhas com hash e salt (por exemplo, bcrypt), nunca em texto puro.

5.3 Considerações sobre a LGPD

Os dados de contato e o CPF de pessoas físicas (tabelas CLIENTE e PESSOA_FISICA) são dados pessoais. Como recomendação, o acesso de leitura a esses dados deve ser restrito à administração e ao atendimento, e não ser concedido aos mecânicos. Atualmente, essa restrição não é aplicada pelo sistema.

6. Gatilhos e Auditoria (B6)

A integridade das informações de histórico, estoque e valor da OS é garantida por triggers do MySQL, acionados automaticamente. Por isso, a aplicação não replica essas regras em código Python.

6.1 Regras Automatizadas

Evento	Comportamento automático
Abertura de OS	Criação do registro inicial em HISTORICO_STATUS_OS.
Alteração de status da OS	Criação de um novo registro de histórico.
UPDATE ou DELETE em HISTORICO_STATUS_OS	Bloqueados: o histórico é imutável.
Inserção de item em ITEM_PECA	Validação do estoque e baixa da quantidade em PECA.
Remoção de item em ITEM_PECA	Devolução da quantidade ao estoque.
Inclusão de itens (ITEM_SERVICO e ITEM_PECA)	Cálculo automático do valor_total da OS; os itens armazenam o valor_cobrado.

6.2 Histórico Automático e Imutável da OS

Cada registro de HISTORICO_STATUS_OS guarda o status anterior, o status novo, a data e hora da mudança, a observação e o mecânico responsável. Como o histórico não pode ser alterado nem excluído, ele funciona como trilha de auditoria do ciclo de vida da OS. Exemplo de consulta ao histórico de uma OS:

```
SELECT status_anterior, status_novo, data_mudanca, observacao
FROM HISTORICO_STATUS_OS
WHERE id_os = %s
ORDER BY data_mudanca;
```

Como evolução futura, pode-se avaliar a auditoria adicional de ajustes de estoque e de alterações de valor_total.

7. Projeto Físico e Armazenamento (B7)

A transição do modelo lógico para a operação física exigiu a revisão dos tipos de dados. As chaves id_os (Ordem de Serviço) e id_historico utilizam o tipo **INT**, cuja capacidade (até 2.147.483.647 registros) é muito superior à necessidade prevista para a oficina.

7.1 Estimativa de Crescimento e Particionamento

Considerando uma média operacional de 40 Ordens de Serviço por dia útil (aproximadamente 10.000 OS por ano), projeta-se um volume de 20.000 OS em dois anos. Como cada OS gera de 3 a 5 eventos de histórico, a tabela HISTORICO_STATUS_OS deve alcançar entre 60.000 e 100.000 registros no mesmo período, crescendo proporcionalmente ao número de OS.

Proposta futura de arquivamento: como as OS perdem relevância operacional diária após os 90 dias de garantia (RN18), propõe-se avaliar o particionamento horizontal de HISTORICO_STATUS_OS por faixa de datas (RANGE), movendo dados antigos para armazenamento de arquivamento (*cold data*). Essa proposta **não está implementada**. No MySQL, o particionamento exigirá a revisão das chaves únicas, pois todas elas (inclusive a chave primária) devem conter a coluna usada na partição; além disso, tabelas InnoDB particionadas não suportam chaves estrangeiras, o que demandará reavaliar os relacionamentos da tabela.

7.2 Plano de Backup Proposto

Escopo	Frequência	Retenção	Recuperação (Restore)
Dump lógico (estrutura e dados)	Diária (às 02:00)	Últimos 15 dias	Restauração semanal em ambiente de homologação para validar o arquivo.
Logs binários (point-in-time)	Contínua	Últimas 48 horas	Recuperação até o instante anterior a uma falha crítica.

8. Declaração de Uso de Inteligência Artificial

Em conformidade com as diretrizes do projeto e com a integridade acadêmica, declaramos o uso assistido de ferramentas de IA (modelos LLM) nesta etapa, estritamente para o seguinte escopo:

- Apoio sintático na construção das consultas parametrizadas em Python.
- Estruturação e revisão gramatical deste documento de entrega.
- Elaboração dos dados sintéticos fictícios para a carga em lote (DML), de modo a gerar massa de testes sem o uso de dados pessoais reais.

Verificação dos Resultados: os trechos de código e as consultas sugeridos pela IA foram revisados manualmente pela equipe, que se responsabiliza pelo conteúdo deste relatório.